

Received 28 October 2024, accepted 29 November 2024, date of publication 5 December 2024,
date of current version 17 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3512434

RESEARCH ARTICLE

Human-in-the-Loop Learning With LLMs for Efficient RASE Tagging in Building Compliance Regulations

DHOYAZAN AL-TURKI¹, HANSI HETTIARACHCHI², MOHAMED MEDHAT GABER¹,
MOHAMMED M. ABDELSAMEA³, SHADI BASURRA¹, SIMA IRANMANESH¹,
HADEEL SAADANY¹, AND EDLIRA VAKAJ¹

¹School of Computing and Digital Technology, Birmingham City University, B4 7RJ Birmingham, U.K.

²School of Computing and Communications, Lancaster University, LA1 4YW Lancaster, U.K.

³Department of Computer Science, University of Exeter, EX4 4PY Exeter, U.K.

Corresponding author: Mohammed M. Abdelsamea (m.abdelsamea@exeter.ac.uk)

This work was supported in part by European Union's Horizon Europe Research and Innovation Program under Agreement 101056973 (ACCORD); and in part by the Horizon Europe Project [ACCORD] are supported by United Kingdom Research and Innovation (UKRI) of Cardiff University under Grant 10040207, Birmingham City University under Grant 10038999, and Building Smart International under Grant 10049977.

ABSTRACT Automated compliance checking (ACC) in the Architecture, Engineering, and Construction (AEC) sector represents a pivotal task which is traditionally executed manually, demanding significant time and labor. This work investigates the automation of the Requirement, Applicability, Selection, and Exception (RASE) methodology for building regulatory compliance through the utilization of Large Language Models (LLMs) and active learning techniques. Specifically, we focus on the development and assessment of a system using the OpenAI GPT-4o model to transmute building regulation texts into structured YAML formats conducive to ACC processes. The study encompasses three experimental paradigms: few-shot learning, fine-tuning learning, and progressive active learning. Initial results from the few-shot learning experiment illustrate the model's preliminary ability to interpret and process regulatory texts with limited examples. Fine-tuning enhances model performance by training it on a specialized dataset, thereby improving structural and textual accuracy. Progressive active learning, by iteratively incorporating expert feedback, further refines the accuracy of the model. The findings demonstrate substantial enhancements in both structural and semantic accuracies of the generated YAML files, underscoring the potential of integrating LLMs with active learning to streamline regulatory compliance automation. The methodologies and results presented here offer a comprehensive framework for advancing future research and practical applications in the domain of automated regulatory compliance.

INDEX TERMS Automated compliance checking, RASE, large language models, active learning.

I. INTRODUCTION

Compliance checking is paramount in the Architecture, Engineering, and Construction (AEC) industry to ensure that building designs meet safety and functionality standards. Historically, this process has been performed manually, rendering it labor-intensive, time-consuming, costly, and prone to errors [18]. To address these limitations, there

has been a substantial shift toward automated compliance checking (ACC), which has been a focus of research over the past five decades [19]. The objective of ACC is to enhance the accuracy and efficiency of compliance checks by automating the identification of pertinent building codes and the verification of design compliance. ACC has evolved significantly with the advent of Building Information Modeling (BIM). A comprehensive review by IntechOpen [5], analysing the ACC research within BIM from 1988 to 2020, underscores the pivotal role of advanced BIM technologies in enhancing

The associate editor coordinating the review of this manuscript and approving it for publication was Ines Domingues¹.

the accuracy and efficiency of ACC processes. This trend has made the integration of digital building permits essential, emphasising the importance of interoperability, data standardization, and stakeholder integration [6]. In general, rule-based model-checking approaches were proven to be more effective in identifying non-compliance issues early in the design process, thus reducing the risk of costly modifications during construction [7]. However, the widespread adoption of machine-processable rules has been challenging due to the requirements for data standardization and the complexities inherent in the textual regulations [8]. Addressing these challenges, the verification of the BIM rule-based model, using the requirement, applicability, selection, and exception (RASE) methodology, has emerged as a promising avenue [7]. RASE is a markup language designed to encapsulate regulatory information from textual documents along with their underlying logic, as detailed in the 'RASE' section below [1]. Primarily, it translates regulatory/normative text into well-defined logical rules while preserving the text's structure and semantics, facilitating automated rule checking. However, to the best of our knowledge, RASE has primarily been employed by domain experts so far as to manually annotate regulatory text to convert them into machine-processable formats/rules. To bridge this gap, we propose leveraging the capabilities of Large Language Models (LLMs) to automate the RASE annotation process in this research. The workflow for "Building Compliance Regulations" typically involves two distinct stages. The first stage, which is the focus of our current research, entails the extraction of regulatory rules from official documents and their transformation into a machine-readable format, specifically YAML in our implementation. This stage is crucial for laying the foundation of automated compliance checking by enabling the structured representation of regulatory requirements. The second stage, which involves the validation of these regulations against a BIM (Building Information Model) document, is indeed a critical component of the broader compliance checking process. However, this paper is primarily concerned with the initial stage—specifically, the methodologies and mechanisms employed for converting regulatory text into a structured format suitable for automation. The integration of this structured data into BIM models for compliance validation falls outside the scope of this paper and is intended for future work where these connections will be explored in detail. LLMs have recently revolutionized natural language processing (NLP), including regulatory text processing, aiding in legal compliance and analysis [9], [11]. Recent studies have demonstrated the potential of LLMs, such as GPT-3.5 and GPT-4, in automated legal classification and compliance with significantly improved accuracy [10]. Handling sparse and noisy data is a critical challenge in machine learning applications, particularly in regulatory environments where high-quality labelled data can be limited. Poulinakis et al. [21] discuss how machine learning techniques, including surrogate modelling,

provide efficient computational alternatives in science and engineering, improving data processing and simulation quality. Their insights are applicable to our context of RASE tagging, where accurate data interpretation under constrained datasets is essential. Similarly, Barbierato and Gatti [22] demonstrate machine learning's robustness in electronics applications, where noisy data are common. These studies inform our human-in-the-loop approach with LLMs, helping to mitigate data sparsity issues while enhancing tagging precision in building compliance regulations.

Following this trend, we propose formulating the RASE annotation of regulatory text as a text generation problem. By taking the regulatory text as input, we generate RASE annotations in a comprehensive YAML format. This format includes text annotations and their associated metadata within the RASE scheme, which are essential for generating the final rules, as further described in 'Materials and Methods'. To evaluate the model's ability to handle such intricate output generation, we experimented with several learning techniques, including few-shot learning, fine-tuning, and active learning. Few-shot learning has shown promise in scenarios with limited labeled data [12], while fine-tuning tends to improve accuracy when domain-specific training is necessary [13]. Active learning techniques, using iterative feedback mechanisms, have been shown to significantly enhance the predictive capabilities of machine learning models [14], [16].

The main contributions of this paper can be summarized as follows:

- 1) We automate the complete RASE annotation process by utilizing the predictive capabilities of LLMs, streamlining rule extraction from regulatory documents.
- 2) We introduce a progressive active learning methodology based on GPT-4o, enhancing the annotation process for RASE building regulations by iteratively refining model performance through targeted feedback and incremental data enrichment.
- 3) We explore the capacity of LLMs to recognize intricate information in regulatory documents, including document structure and nested rules, using various learning techniques.
- 4) We propose an automatic evaluation procedure to assess prediction accuracy, focusing on both structural and semantic information crucial for rule extraction.

Rase: The RASE scheme is built upon four primary operators: (1) R – requirement, (2) A – applicability, (3) S – selection, and (4) E – exception. Requirements are the checks that must be satisfied, typically indicated by imperatives such as 'must' and 'shall'. Applicability defines the circumstances under which the check should be applied, thereby restricting its scope. Each check is associated with at least one applicability. In contrast, selection broadens the scope for a check, while exceptions delineate conditions

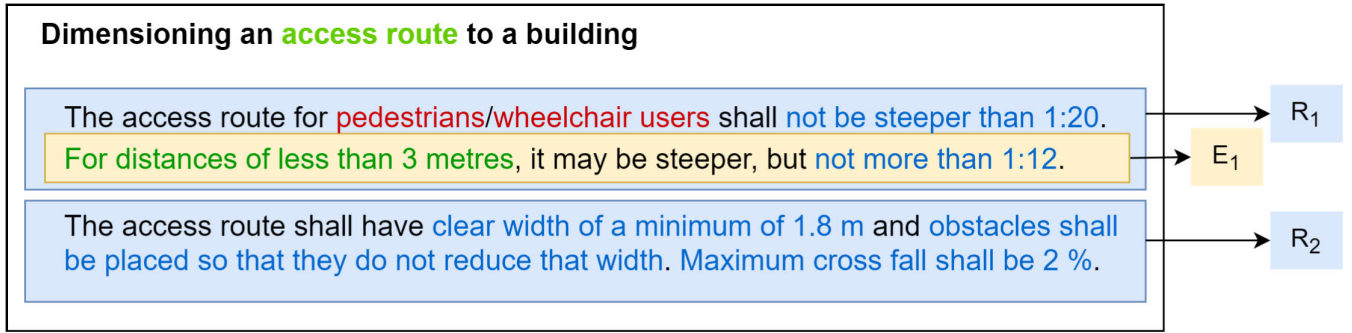


FIGURE 1. RASE annotated example. The color code (in both text and text boxes) represents different RASE elements, where blue is for Requirement, green is for Applicability, red is for Selection, and yellow is for Exception. Colored text boxes represent the outer tags, while colored text inside them represents the inner tags when nested tags are available.

TABLE 1. RASE fine-granular annotations using OTCVU scheme, defining the type, object, target, comparator, value, and unit for each annotation.

Text Phrase	RASE Type	Object	Target	Comparator	Value	Unit
access route	applicability	space	type	=	access_route	
pedestrians	selection	access_route	is_used_by_pedestrians	=	True	
wheelchair users	selection	access_route	is_used_by_wheelchair_users	=	True	
not be steeper than 1:20	requirement	access_route	slope	>	1:20	
distances of less than 3 metres	applicability	access_route	length	<	3	m
not more than 1:12	requirement	access_route	slope	>	1:12	
clear width of a minimum of 1.8 m	requirement	access_route	width	>	1.8	m
obstacles shall be placed so that they do not reduce that width	requirement	access_route	obstructed	=	False	
Maximum cross fall shall be 2 %	requirement	access_route	cross_fall	≤	2%	

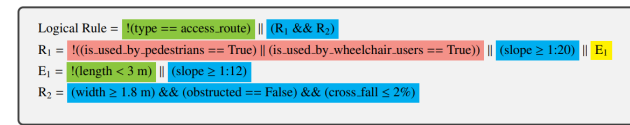


FIGURE 2. This figure presents the logical rule for determining the dimensions of an access route. It defines conditions (R1 and R2) for access routes used by pedestrians and wheelchair users, incorporating exceptions (E1) for short distances. The rule ensures compliance with slope, width, obstruction, and cross-fall requirements.

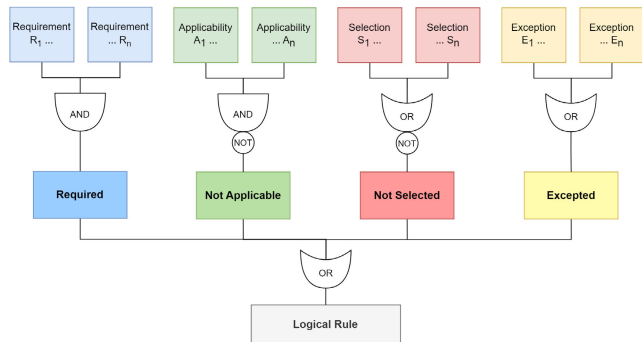


FIGURE 3. Logical structure of the RASE methodology, detailing how requirements (R), applicability (A), selection (S), and exceptions (E) are combined. Requirements are connected via logical AND, while applicability and selection use NOT and OR operators respectively. Exceptions are combined using OR, with the final logical rule integrating all these components to determine the compliance status.

under which the check should be excluded. In general, requirements and applicabilities are common in regulatory texts, whereas selections and exceptions are relatively rare.

The RASE methodology for generating machine-processable rules comprises three primary steps. The first step involves adding markup to text based on the four RASE operators, adhering to text semantics. As shown in Figure 1, text segments of building works often contain nested tags that illustrate the clausal structure of the document. Each requirement block (R_1 and R_2 in Figure 1) represents individual checks composed of the corresponding inline tag sets. The RASE markup primarily captures the structure and arrangement of rules expressed in the text. Subsequently, each fine-grained annotation must be paired with its corresponding metadata to facilitate the generation of logical rules. This second step uses the OTCVU scheme, which identifies the object, target/property, comparator, value, and unit associated with each annotated text phrase. Typically, objects and targets are derived from a domain-specific dictionary or ontology. A target represents an attribute or characteristic of an object. The value, with or without a unit depending on its nature, is checked against the target. For numerical values, the comparator may be less, less-equal, equal, or converse. Comparators for descriptive values are limited to equal, includes, and their converses. Table 1 summarises the fine-granular RASE annotations for the sample in 1. Finally, to transform the operators into rules using Boolean logic, RASE utilizes a specific logical structure illustrated in Figure 3. All requirements are connected by logical AND/conjunction, while all exceptions are combined using logical OR/disjunction. Conversely, negated AND and OR connect the applicabilities and

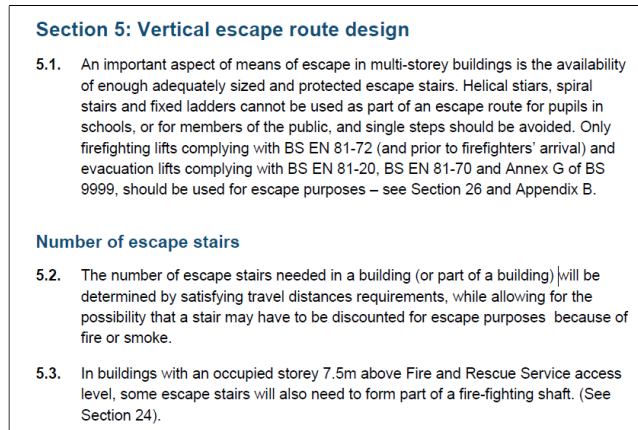


FIGURE 4. Text snippet from England's building regulation: Fire Safety Design for Schools—Building Bulletin 100. These are part of one of the input files for the experiments.

selections according to their semantic structure. These sub-elements are then combined using logical OR to generate the final machine-processable rule. This logical structure allows RASE annotations to be converted into a logical rule (shown in Figure 2) that an intelligent agent or machine can automatically verify. Overall, RASE employs a generic annotation and transformation approach, applicable across various subdomains within the AEC sector. Despite its straightforward design, it has proven remarkably versatile in capturing diverse rule types in regulatory text [1]. Although the final step of the RASE methodology can be performed automatically following the predefined logical structure, the initial two steps require significant domain expertise and meticulous and time-consuming analysis. These steps are crucial, as they lay the foundation for accurately capturing information presented in the text, enabling effective rule generation and ACC.

II. DATASET

The foundation of this investigation was based on a robust dataset comprising 25 detailed text files containing regulatory information on buildings. These documents were meticulously annotated by domain experts to create an accurate baseline dataset for training and evaluating the models. This preparation was essential to ensure the integrity and reliability of the input data, which directly affects the quality of the output.

Here, we used the data generated as part of the Automated Compliance Checks for Construction, Renovation, or Demolition Works (ACCORD) project [4]. To the best of our knowledge, this dataset represents the initial and sole annotated dataset in the Building Compliance Rule Language (BCRL) format. It was thoroughly curated by domain experts after a rigorous annotation process.

Three regulatory/approved building documents from England were used as the basis for creating our dataset: (1) M2: Access to and use of buildings other than dwellings, (2) L2A: Conservation of fuel and power in new buildings

other than dwellings, and (3) BB100: Fire safety design for schools. A total of 25 sections of these documents were carefully annotated using the complete RASE methodology (Section I), and the annotations were structured into YAML files in BCRL format. Since our research focuses exclusively on textual data, we excluded tables and figures from these sections to prepare the data for experimentation (Figure 4 is an example of England's building regulation). The distribution of the RASE categories in the final dataset is presented in Table 2. Notably, the majority of tags are requirements, while exceptions constitute a minority. Selections slightly outnumber exceptions, unlike applicabilities, which are predominantly associated with requirements.

Additionally, Figure 5 illustrates the sequence length distribution of text belonging to each RASE category after removing outliers (e.g., extremely long annotated text sequences). It is evident that both applicabilities and selections are constrained to sequences of up to 8 tokens/words. In contrast, requirements and exceptions exhibit a wide range of sequence lengths, comprising short sequences of a few tokens and longer sequences exceeding 40 tokens. The nested structure of RASE annotations accounts for this variability in sequence lengths. These statistics emphasize the varied characteristics of annotations even within a single category.

A. DATA FORMATTING

The format of output data to be predicted by a machine learning model plays a pivotal role in the model's learning process and performance within a specific task. Thus, following our goal of automating the RASE annotation process, our initial step required the design of a data format capable of incorporating RASE text annotations along with their metadata. It was crucial to ensure simplicity in the format without any ambiguities to facilitate effective model learning. Moreover, the output needed to be informative, covering all the details essential for automatic rule generation to support ACC. However, given that RASE was originally designed as a manual approach for domain experts, it lacks an established machine-processable format capable of capturing all annotated information. To overcome this limitation, recent research conducted as part of the Automated Compliance Checks for Construction, Renovation, or Demolition Works (ACCORD) project¹ introduced a YAML-based data format named Building Compliance Rule Language (BCRL) [4]. This format was designed to capture all RASE annotations comprehensively. YAML is a human-friendly and machine-processable data serialization language [3]. It also provides a consistent model compatible with various programming languages, making it well suited for automatic rule generation from building regulations within a wider scope. Besides capturing RASE data, BCRL also preserves document and text clausal structures, enhancing the informativeness of the output. In addition, BCRL maintains a direct mapping

¹ ACCORD project's website-<https://accordproject.eu/>

TABLE 2. RASE category distribution over the annotated document sections. Columns R, A, S, and E represent the requirement, applicability, selection, and exception counts.

Approved Document	Section	R	A	S	E
M2: Access to and use of buildings other than dwellings	0: General guidance	0	0	0	0
	1: Access to buildings other than dwellings	184	44	8	10
	2: Access into buildings other than dwellings	134	45	2	4
	3: Horizontal and vertical circulation in buildings other than dwellings	345	100	24	24
	4: Facilities in buildings other than dwellings	210	79	22	26
L2A: Conservation of fuel and power in new buildings other than dwellings	5: Sanitary accommodation in buildings other than dwellings	304	106	40	12
	0: Conservation of fuel and power in new buildings other than dwellings	6	2	0	1
	1: The requirements	0	0	0	0
	2: Design standards	128	48	16	10
	3: Quality of construction and commissioning	93	22	2	10
BB100: Fire safety design for schools	4: Providing information	12	2	0	0
	5: Model designs	0	0	0	0
	4: Horizontal escape route design	121	46	14	13
	5: Vertical escape route design	166	69	13	19
	6: General provisions for means of escape	165	58	25	15
	8: Wall and ceiling linings	0	0	0	0
	9: Variations and special provisions	33	15	3	2
	10: Thermoplastic materials	26	15	6	6
	11: Loadbearing elements of structure	2	2	0	0
	12: Compartmentation	4	1	0	0
	13: Construction of compartment walls and floors	95	41	16	10
	14: Cavities	83	23	16	31
	15: Protection of openings and fire-stopping	72	21	15	13
	26: Inclusive design	63	11	6	4
	33: Regulation 38	15	1	2	0
Total		2261	751	230	210

between the original regulatory text and the annotated data to facilitate more effective human-in-the-loop review processes.

The text-to-BCRL conversion comprises two stages. The first stage transforms text from a document to the BCRL format by extracting only the document's structure and context. This primarily captures the organization of the content within the document, including sections, titles, paragraphs, bullet points, and the text itself. These elements are arranged in a nested format in YAML, reflecting their hierarchical structure in the document. Each element is assigned a unique identifier that denotes its position within the document. In addition, they are tagged with a type (i.e., DocumentSubdivision or Statement) to indicate their category within the structural breakdown.

The next stage implements the RASE methodology to populate the elements in the YAML format with RASE annotations. RASE tags and their associated metadata are incorporated into the YAML elements or their nested breakdowns. The element named 'isOperationalizedBy' includes all the metadata of fine-granular tags, which constitute logical rules. In addition to the standard RASE metadata elements outlined in Section I, BCRL introduces the method type attribute, which indicates the appropriate check method to be used with the corresponding rule, thus simplifying the rule generation process. Common method types include 'BooleanCheckMethod', 'NumericalCheckMethod', and 'CategoryCheckMethod'. More details on additional categories are provided in the BCRL documentation [4].

III. METHODOLOGY

This section delineates the methodologies employed in automating the annotation of regulatory texts using the

RASE scheme. Utilizing the capabilities of Large Language Models (LLMs), this study focused on transforming textual descriptions of building regulations into structured YAML formats, which are instrumental in facilitating automated compliance check (ACC) processes. The main focus is on "text" as the primary concern was text tagging within the context of regulatory compliance. However, we acknowledge the importance of addressing other elements such as tables and images. In our approach, these elements are represented in a machine-understandable format within the YAML file. For instance, images are encoded as Base64 text within the YAML structure, whilst tables are tagged using a "Table" tag.

The study employs three distinct experiments utilizing OpenAI's GPT-4o model, aimed at exploring the potential of few-shot, fine-tuning, and fine-tuning with progressive active learning techniques in processing regulatory texts. We selected OpenAI's GPT-4o for our study because it is currently the largest and most advanced LLM available, with around 1.76 trillion hyperparameters. This substantial capacity allows for more nuanced and accurate processing of complex regulatory texts, which is critical for the success of our approach.

A. FEW-SHOT LEARNING WITH GPT-4O

Our few-shot model utilizes GPT-4o, an optimized variant of OpenAI's third-generation language models, offering improved processing speed and efficiency due to its advanced architecture. This approach enables the model to generalize from a few examples to new input without extensive traditional training. In this setting, we selected three examples from the dataset as a few-shot prompt providing the model

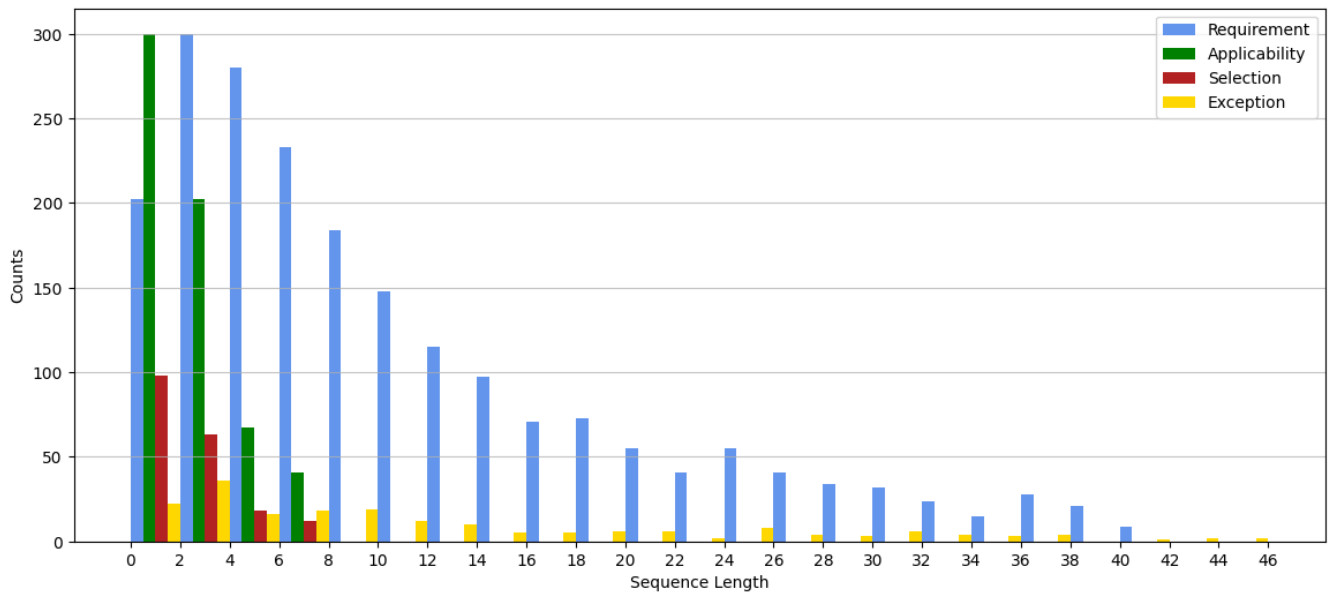


FIGURE 5. Sequence length distribution of each RASE tag within the complete dataset.

with clear input-output pairs to establish the task context, with the input being the text file content and the output being the corresponding YAML file. The model was then tested on new unseen text files tasked with converting these texts into YAML format using the context learned from the few-shot examples. The evaluation criteria focused on the model’s ability to recognize and convert intricate information in regulatory documents, including document structure and nested rules, with both structural and semantic accuracies assessed to ensure the fidelity of the YAML conversion. Algorithm 1 illustrates the process in detail.

B. FINE-TUNING WITH GPT-4O

Our second attempt involves fine-tuning the GPT-4o model [20], allowing customization to specific tasks by training on a targeted dataset. OpenAI provides a fine-tuning service that allows users to fine-tune their models, including GPT-4, on custom datasets. This process involves uploading a dataset that aligns with the format and structure required by the OpenAI API, after which the model is fine-tuned based on the provided data. The fine-tuning process allows users to customise the model’s behaviour for specific tasks by training it on a specialised dataset. OpenAI manages the underlying infrastructure, ensuring that the fine-tuning process is efficient and integrates seamlessly with the pre-existing model architecture.

For data preparation, we utilized 25 paired text and YAML files derived from building regulations, selecting 15 pairs for training, 5 pairs for validation, and reserving 5 pairs for testing. We created two JSONL files, training.jsonl and validation.jsonl, for the fine-tuning process, formatting each entry to include the system’s role, user input (text file

content), and assistant output (YAML file content). The fine-tuning process involved initiating a fine-tuning job on OpenAI’s server using the prepared JSONL files, training the model on the training.jsonl file. The fine-tuned model was subsequently tested on the five remaining text-YAML file pairs to evaluate its performance. Detailed results and analysis of this testing phase are discussed in the “Model Evaluation” section. The process of the fine-tuning has been illustrated in Algorithm 2.

C. GPT-4O FINE-TUNING WITH PROGRESSIVE ACTIVE LEARNING

Here, we explored the effectiveness of combining fine-tuning with progressive active learning to improve the performance of GPT-4o in generating YAML files from text-based building regulations. This approach aims to iteratively enhance the model’s accuracy by incorporating human expertise into the learning process. The model we employed is the GPT-4o variant, fine-tuned using a dataset specifically tailored to our domain. The combination of fine-tuning with progressive active learning allows the model to iteratively improve by incorporating human feedback into its learning process. The fine-tuning process leverages the standard supervised learning framework, where the model is trained on a specialised dataset, adjusting the weights based on the cross-entropy loss function. This function, commonly used in text generation tasks, measures the divergence between the predicted output and the reference output, guiding the gradient descent process to minimise this divergence. In terms of integrating active learning with GPT-4o, we adopted a progressive active learning approach. After the initial fine-tuning, the model’s predictions were evaluated for accuracy.

Algorithm 1 Few-Shot Learning With GPT-4o for YAML Conversion

```

1: procedure FEWSHOTLEARNING(Examples, Model)
2:   FewShotPrompts  $\leftarrow$  SELECTEXAMPLES(Examples)
3:   TestData  $\leftarrow$  SELECTUNSEENTEXTFILES
4:   EvalResults  $\leftarrow$  TESTMODEL(Model, FewShotPrompts, TestData)
5:   return EvalResults
6: end procedure
7: function SELECTEXAMPLES(Examples)
8:   FewShotPrompts  $\leftarrow$  []
9:   for each example in Examples do
10:    FewShotPrompt  $\leftarrow$  format example as input-output pair
11:    append FewShotPrompt to FewShotPrompts
12:   end for
13:   return FewShotPrompts
14: end function
15: function TESTMODEL(Model, FewShotPrompts, TestData)
16:   EvaluationResults  $\leftarrow$  []
17:   for each testItem in TestData do
18:    prediction  $\leftarrow$  GENERATEOUTPUT(Model, FewShotPrompts, testItem.input)
19:    evaluation  $\leftarrow$  EVALUATEPREDICTION(prediction, testItem.output)
20:    append evaluation to EvaluationResults
21:   end for
22:   return EvaluationResults
23: end function
24: function EVALUATEPREDICTION(prediction, expectedOutput)
25:   return comparison of prediction with expectedOutput
26: end function

```

High-confidence predictions, as determined by a predefined accuracy threshold, were selected for further refinement through expert review. These reviewed predictions were then incorporated back into the training dataset, enabling the model to learn from its previous mistakes and refine its predictions in subsequent iterations.

1) INITIAL DATASET AND MODEL FINE-TUNING

- 1) *Initial Fine-Tuning*: We started with 25 paired files consisting of text input and corresponding YAML output. For the initial fine-tuning, we selected 5 pairs for training and 2 pairs for validation. This selection was aimed at providing the model with a foundational understanding of the task while ensuring a balance between training and validation to prevent overfitting.
- 2) *Initial Validation*: After the initial fine-tuning, the model was tested on the remaining 18 pairs. Each predicted YAML file was evaluated for both structural and content accuracy based on the evaluation metrics discussed in “Results and Discussion” section.

2) ACCURACY EVALUATION AND CONFIDENCE POOLING

- 3) *Accuracy Assessment*: Each of the 18 predicted YAML files was assessed for both structure and text accuracy (as mentioned in the Structural Similarity and Text Similarity subsections). Files with an accuracy rate of

70% or higher were classified into a High Confidence pool, while those with lower accuracy were placed in a Low Confidence pool. This threshold was chosen to ensure that only reasonably accurate predictions were considered for further refinement.

- 4) *Expert Review*: Files in the High Confidence pool were then reviewed by a human expert. The expert’s role was to correct any remaining errors and enhance the overall quality of the YAML outputs. This human-in-the-loop approach ensured that the high-confidence predictions met the necessary standards before being used for further training.

3) ITERATIVE FINE-TUNING

- 5) *Incorporating Expert Feedback*: The modified high-confidence files, now corrected by the expert, were added back into the training dataset. This augmented dataset was used to fine-tune the GPT-4o model again. By incorporating expert corrections, the model could learn from its mistakes and improve its future predictions.
- 6) *Reiteration Process*: This process of fine-tuning, predicting, evaluating, and expert review was repeated iteratively. Each iteration aimed to progressively improve the model’s performance by continually refining the dataset with high-quality, expert-verified examples.

Algorithm 2 Fine-Tuning GPT-4o for YAML Conversion

```

1: procedure FINETUNEMODEL(Data, Model)
2:   TrainingData  $\leftarrow$  select 15 pairs from Data
3:   ValidationData  $\leftarrow$  select 5 pairs from Data
4:   TestData  $\leftarrow$  select 5 pairs from Data
5:   TrainingJSONL  $\leftarrow$  CONVERTTOJSONL(TrainingData)
6:   ValidationJSONL  $\leftarrow$  CONVERTTOJSONL(ValidationData)
7:   upload TrainingJSONL and ValidationJSONL to OpenAI API
8:   FineTunedModel  $\leftarrow$  CREATEFINETUNINGJOB(Model, TrainingJSONL, ValidationJSONL)
9:   train FineTunedModel using OpenAI API
10:  EvalResults  $\leftarrow$  EVALUATEMODEL(FineTunedModel, TestData)
11:  return EvalResults
12: end procedure
13: function CONVERTTOJSONL(Data)
14:   for each item in Data do
15:     JSONLItem  $\leftarrow$  {"messages" : [{"role" : "system", "content" : "RASEContext"}, {"role" :
"user", "content" : item.text}, {"role": "assistant", "content": item.yaml}]}
16:     append JSONLItem to JSONLData
17:   end for
18:   return JSONLData
19: end function
20: function EVALUATEMODEL(Model, TestData)
21:   for each testItem in TestData do
22:     prediction  $\leftarrow$  GENERATEOUTPUT(Model, testItem.text)
23:     compare prediction with testItem.yaml
24:   end for
25:   return evaluation results
26: end function

```

7) *Completion Criterion*: The iterative process continued until all 23 pairs (excluding the initial 2 validation pairs) were incorporated into the training dataset. This approach ensured that the model was trained on a comprehensive set of examples, each having been subjected to rigorous quality control.

The details of this process are illustrated in Figure 6 and Algorithm 3.

D. INCREMENTAL PREDICTION STRATEGY

All experiments incorporated an incremental prediction strategy to overcome the token generation limits inherent to GPT models, which are capped at 4096 tokens per generation. This strategy was essential for handling lengthy documents, typical of regulatory texts, that exceeded the model's output capacity. The incremental prediction approach is detailed as follows:

- 1) **Initial Input Processing**: The model begins by processing the initial segment of the regulatory text. This initial input is kept within the token limit, ensuring that the model can generate an output without truncation.
- 2) **Partial YAML Generation**: Based on the initial input, the model generates a portion of the YAML output. This partial YAML segment fits within the token limits and represents a fraction of the complete conversion.

- 3) **Context Reinsertion**: The generated YAML segment is reinserted into the model's context. This updated context includes both the initial regulatory text segment and the corresponding generated YAML.
- 4) **Subsequent Input Segmentation**: The next segment of the regulatory text, following the initial segment, is appended to the updated context. Care is taken to ensure that the combined context (previous context plus the next segment) remains within the token limit.
- 5) **Iterative Generation**: The model is prompted again to continue the YAML generation from where it left off. This iterative process involves repeatedly generating new segments of the YAML output, reinserting them into the context, and appending subsequent text segments until the entire document is processed.
- 6) **Handling Long Documents**: For particularly lengthy documents, this incremental approach ensures that the entire text is eventually converted into YAML format, with each segment processed and reintegrated methodically. This avoids issues related to token overflow and ensures continuity in the generated output.
- 7) **Evaluation of Continuity and Accuracy**: At each step, the generated YAML segments are evaluated for continuity and accuracy to ensure that the

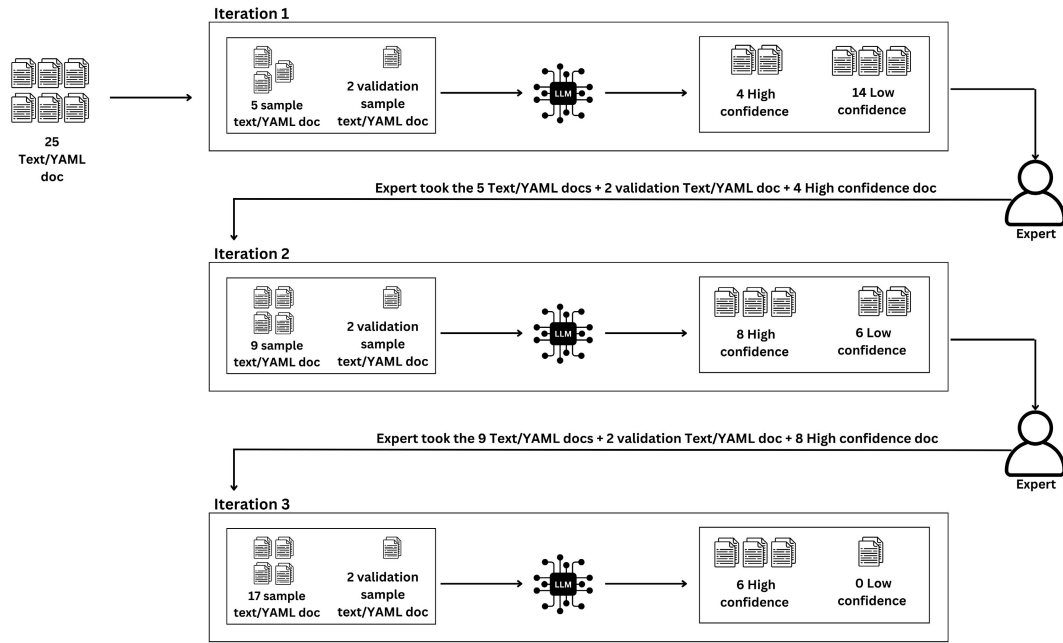


FIGURE 6. The progressive active learning process.

incremental predictions align seamlessly with the previously generated segments. This evaluation helps maintain the structural and semantic integrity of the output.

Algorithm 4 explains the process of the incremental prediction.

IV. RESULTS AND DISCUSSION

Here, we present and discuss the results obtained from three experiments conducted to evaluate the performance of our system in generating YAML files from regulatory text. The evaluation is grounded in two primary metrics: structural similarity and text similarity, which are proposed and employed to comprehensively assess both the format and content of the generated YAML files.

A. STRUCTURAL SIMILARITY

The structural accuracy of the generated YAML files is evaluated using graph similarity metrics. Each YAML file is converted to a graph representation where the nodes represent keys and values, and the edges represent the hierarchical relationships between these elements. We employ the SimGNN model [17], a neural network-based approach, to compute the similarity between the generated and reference graphs. This process begins with converting the generated YAML and the reference YAML into graph structures. Initial node embeddings are then computed using Graph Convolutional Networks (GCNs), which aggregate information from neighboring nodes to produce a rich representation of the graph's structure. These node embeddings are further aggregated into a single graph-level embedding using a global context-aware

attention mechanism, which computes a global context vector for the graph and uses it to weigh the importance of each node embedding. Interaction scores between graph-level embeddings are computed using a Neural Tensor Network (NTN). Additionally, pairwise similarity scores between nodes of the two graphs are computed, and histogram features are extracted from these scores. Finally, graph-level interaction scores and histogram features are combined using fully connected layers to produce a final similarity score.

To quantify the structural differences between the generated and reference graphs, we compute the Graph Edit Distance (GED), which measures the minimum number of operations (node/edge insertions, deletions, and relabelings) required to transform one graph into the other. The GED is normalized and transformed into a similarity score ranging from 0 to 1 using the exponential function $\lambda(x) = e^{-x}$, where x is the normalized GED.

B. TEXT SIMILARITY

The similarity of text between the content of the generated and reference YAML files is evaluated using accuracy and adjusted accuracy metrics. The accuracy metric measures the overlap of words between the two texts, while the adjusted accuracy accounts for differences in chunk sizes to penalize discrepancies in text segmentation. First, the number of overlapping words between the generated and reference texts is calculated to determine the common words. Then, the total number of unique words in both combined texts is calculated. The basic accuracy is calculated as the ratio of common words

Algorithm 3 GPT-4o Fine-Tuning With Progressive Active Learning for YAML Conversion

```

1: procedure PROGRESSIVEFINETUNEMODEL(Data, Model)
2:   InitTrain  $\leftarrow$  5 pairs from Data, InitValid  $\leftarrow$  2 pairs from Data, RemainData  $\leftarrow$  18 pairs from Data
3:   TrainingJSONL  $\leftarrow$  CONVERTTOJSONL(InitialTrainingData)
4:   ValidationJSONL  $\leftarrow$  CONVERTTOJSONL(InitialValidationData)
5:   FineTunedModel  $\leftarrow$  CREATEFINETUNINGJOB(Model, TrainingJSONL, ValidationJSONL)
6:   train FineTunedModel using OpenAI API
7:   HighConfidencePool  $\leftarrow$  {}
8:   LowConfidencePool  $\leftarrow$  {}
9:   InitialEvalResults  $\leftarrow$  EVALUATEMODEL(FineTunedModel, RemainingData)
10:  for each result in InitialEvalResults do
11:    if result.accuracy  $\geq$  70% then
12:      add result to HighConfidencePool
13:    else
14:      add result to LowConfidencePool
15:    end if
16:  end for
17:  EXPERTREVIEW(HighConfidencePool)
18:  AugmentedTrainingData  $\leftarrow$  merge InitialTrainingData and HighConfidencePool
19:  repeat
20:    TrainingJSONL  $\leftarrow$  CONVERTTOJSONL(AugmentedTrainingData)
21:    FineTunedModel  $\leftarrow$  CREATEFINETUNINGJOB(Model, TrainingJSONL)
22:    train FineTunedModel using OpenAI API
23:    EvalResults  $\leftarrow$  EVALUATEMODEL(FineTunedModel, RemainingData)
24:    HighConfidencePool  $\leftarrow$  {}
25:    LowConfidencePool  $\leftarrow$  {}
26:    for each result in EvalResults do
27:      if result.accuracy  $\geq$  70% then
28:        add result to HighConfidencePool
29:      else
30:        add result to LowConfidencePool
31:      end if
32:    end for
33:    EXPERTREVIEW(HighConfidencePool)
34:    AugmentedTrainingData  $\leftarrow$  merge AugmentedTrainingData and HighConfidencePool
35:  until all pairs in RemainingData are incorporated into AugmentedTrainingData
36:  return final FineTunedModel
37: end procedure
38: function CONVERTTOJSONL(Data)
39:  for each item in Data do
40:    JSONLItem  $\leftarrow$  {"messages": [{"role": "system", "content": "RASEContext"}, {"role": "user", "content": item.text}, {"role": "assistant", "content": item.yaml}]}
41:    append JSONLItem to JSONLData
42:  end for
43:  return JSONLData
44: end function
45: function EVALUATEMODEL(Model, TestData)
46:  for each testItem in TestData do
47:    prediction  $\leftarrow$  GENERATEOUTPUT(Model, testItem.text)
48:    compare prediction with testItem.yaml
49:    calculate accuracy of prediction
50:  end for
51:  return evaluation results
52: end function
53: function EXPERTREVIEW(Data)
54:  for each item in Data do
55:    review and correct item
56:  end for
57: end function

```

to total unique words, defined as

$$\text{accuracy}(x, y) = \frac{\text{common}(x, y)}{\text{total}(x, y)}.$$

To account for text segmentation differences, we determine the number of chunks in both the generated (c_x) and reference (c_y) texts. The chunk penalty is then computed using the logarithmic difference in the chunk counts, normalized by the

Algorithm 4 Incremental Prediction Strategy for YAML Conversion

```

1: procedure INCREMENTALPREDICTION(Text, Model)
2:   Context  $\leftarrow \emptyset$ 
3:   Segments  $\leftarrow$  SEGMENTTEXT(Text)
4:   YAML  $\leftarrow \emptyset$ 
5:   for each segment in Segments do
6:     Context  $\leftarrow$  Context + segment
7:     if TOKENCOUNT(Context) > 4096 then
8:       Context  $\leftarrow$  TRUNCATE(Context)
9:     end if
10:    PartialYAML  $\leftarrow$  GENERATEYAML(Model, Context)
11:    YAML  $\leftarrow$  YAML + PartialYAML
12:    Context  $\leftarrow$  Context + PartialYAML
13:  end for
14:  return YAML
15: end procedure
16: function SEGMENTTEXT(Text)
17:   Segments  $\leftarrow$  SPLIT(Text, MaxSegmentLength)
18:   return Segments
19: end function
20: function TOKENCOUNT(Context)
21:   return COUNTTOKENS(Context)
22: end function
23: function TRUNCATE(Context)
24:   return REMOVEOLDESTSEGMENTS(Context, MaxTokenLimit)
25: end function
26: function GENERATEYAML(Model, Context)
27:   PartialYAML  $\leftarrow$  MODELGENERATE(Model, Context)
28:   return PartialYAML
29: end function

```

total number of chunks in the larger text (t). This is defined in Equation 1:

$$\text{chunk penalty}(c_x, c_y, t) = \frac{\log_2(\text{chunk difference}(c_x, c_y) + 1)}{\log_2(t + 1)}. \quad (1)$$

Finally, the adjusted accuracy is calculated to incorporate the chunk penalty, reducing the impact of segmentation differences. The formula for adjusted accuracy is shown in Equation 2:

$$\begin{aligned} &\text{adjusted accuracy}(x, y, c_x, c_y, t, w) \\ &= \text{accuracy}(x, y) \times (1 - w \times \text{chunk penalty}(c_x, c_y, t)), \end{aligned} \quad (2)$$

where w is the chunk penalty weight.

C. EXPERIMENT 1: FEW-SHOT LEARNING WITH GPT-4O

In the first experiment, we utilized a few-shot learning approach with GPT-4o by providing the model with three sample pairs as prompts. The model was then tasked with predicting YAML structures for 22 test samples. The average structural similarity score was 0.69, while the average text similarity score was 0.76. This higher text similarity score

compared to structural similarity suggests that although the model was relatively effective in capturing the content, it struggled more to maintain the correct hierarchical structure. Table 3 presents the minimum, maximum, standard deviation, and average scores for both text and structural similarity.

TABLE 3. Few-Shot experiment summary of text and structural similarity accuracy scores.

	Min	Max	Standard Deviation	Average
Structural Similarity	0.57	0.79	0.065	0.69
Text Similarity	0.68	0.85	0.048	0.76

D. EXPERIMENT 2: FINE-TUNING WITH GPT-4O

The second experiment involved fine-tuning the GPT-4o model using a dataset of 25 paired text and YAML files, with 15 pairs for training, 5 pairs for validation, and 5 pairs for testing. The structural and text similarity results for this experiment are presented in Table 4. The improvement over the few-shot learning approach indicates that fine-tuning allows the model to better learn and replicate the hierarchical

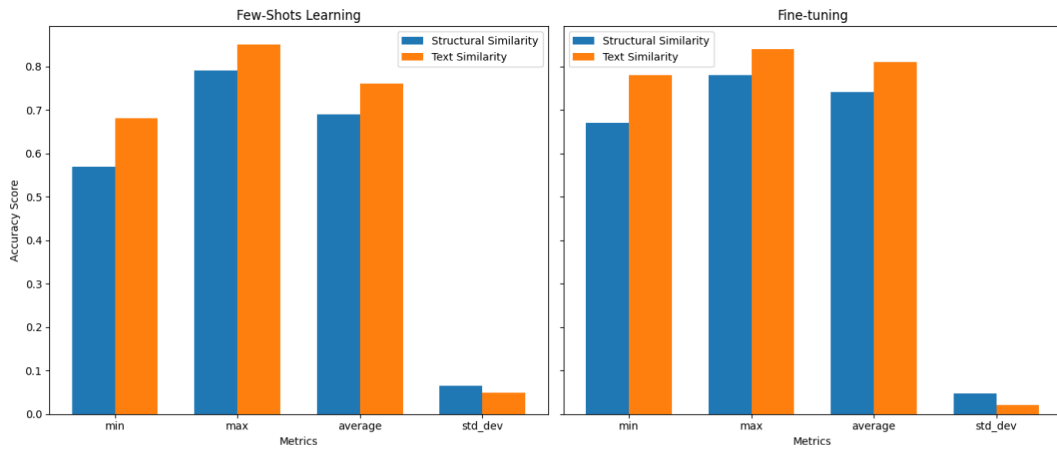


FIGURE 7. Structure and text similarity accuracy scores for few-shot learning and fine-tuning experiments.

structures in the YAML files. Figure 7 show the accuracy scores for both few-shot learning and fine-tuning.

TABLE 4. Fine-Tuning experiment summary of text and structural similarity accuracy scores.

	Min	Max	Standard Deviation	Average
Structural Similarity	0.67	0.78	0.047	0.74
Text Similarity	0.78	0.84	0.02	0.81

E. EXPERIMENT 3: GPT-4O FINE-TUNING WITH PROGRESSIVE ACTIVE LEARNING

The third experiment combined fine-tuning with progressive active learning. We began with an initial training set of 5 pairs and a validation set of 2 pairs. The model then predicted the remaining 18 samples. Predictions with structural and text similarity scores above 0.7 were classified into a high-confidence pool, while others were placed in a low-confidence pool. This process iterated through three cycles, as detailed in Table 5 and Figure 8.

TABLE 5. Progressive active learning iterations.

Iter	Train Samples	Val Samples	High Conf	Low Conf
Initial	5	2	4	14
Second	9	2	8	6
Third	17	2	6	0

Following the completion of the third iteration, the model was trained and subsequently evaluated on a set of 25 test files. The results of this evaluation are presented in Table 6.

The iterative process led to a progressive increase in the number of high-confidence samples, demonstrating the effectiveness of incorporating human feedback and progressively fine-tuning the model. This approach ensures that the model learns from its mistakes and continuously

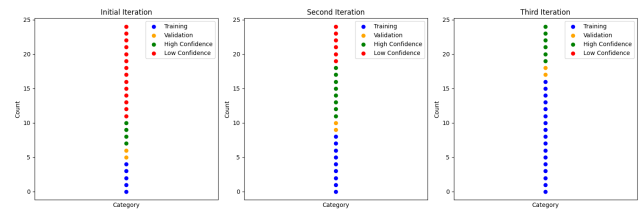


FIGURE 8. Scatter plots for progressive active learning iterations. The plots show the counts of training samples (blue), validation samples (orange), high confidence samples (green), and low confidence samples (red) for each iteration.

TABLE 6. Progressive active learning experiment summary of text and structural similarity accuracy scores.

	Min	Max	Standard Deviation	Average
Structural Similarity	0.74	0.88	0.03	0.83
Text Similarity	0.77	0.93	0.05	0.87

improves its performance. Figure 9 and 10 illustrate a comparative analysis of structure and text similarities across three different learning methods: few-shot learning, fine-tuning, and active learning. For each method, two metrics are evaluated: structural similarity and text similarity. The x-axis represents the sample indices, while the y-axis denotes the similarity scores. The results indicate that, across all sample indices, the active learning method generally achieves higher similarity scores compared to both few-shot learning and fine-tuning, particularly in text similarity. Fine-tuning shows a noticeable improvement over few-shot learning, particularly in structural similarity, but does not reach the levels attained by active learning. The close proximity of the bars within each method category suggests consistent performance across the samples, with active learning demonstrating the most robust and reliable performance in automating the RASE annotation process. This figure effectively highlights the superior performance of the active learning approach, particularly in its ability to enhance both structural and textual

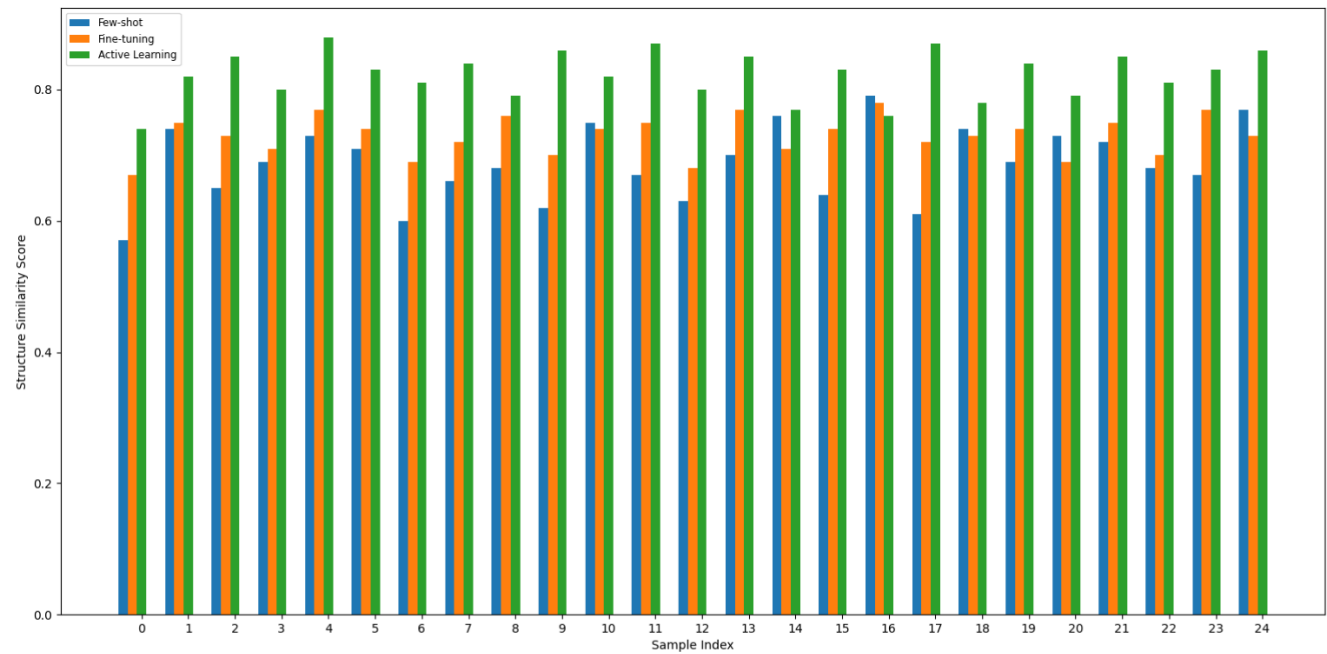


FIGURE 9. Comparing the structure similarity accuracy scores for the different learning methods.

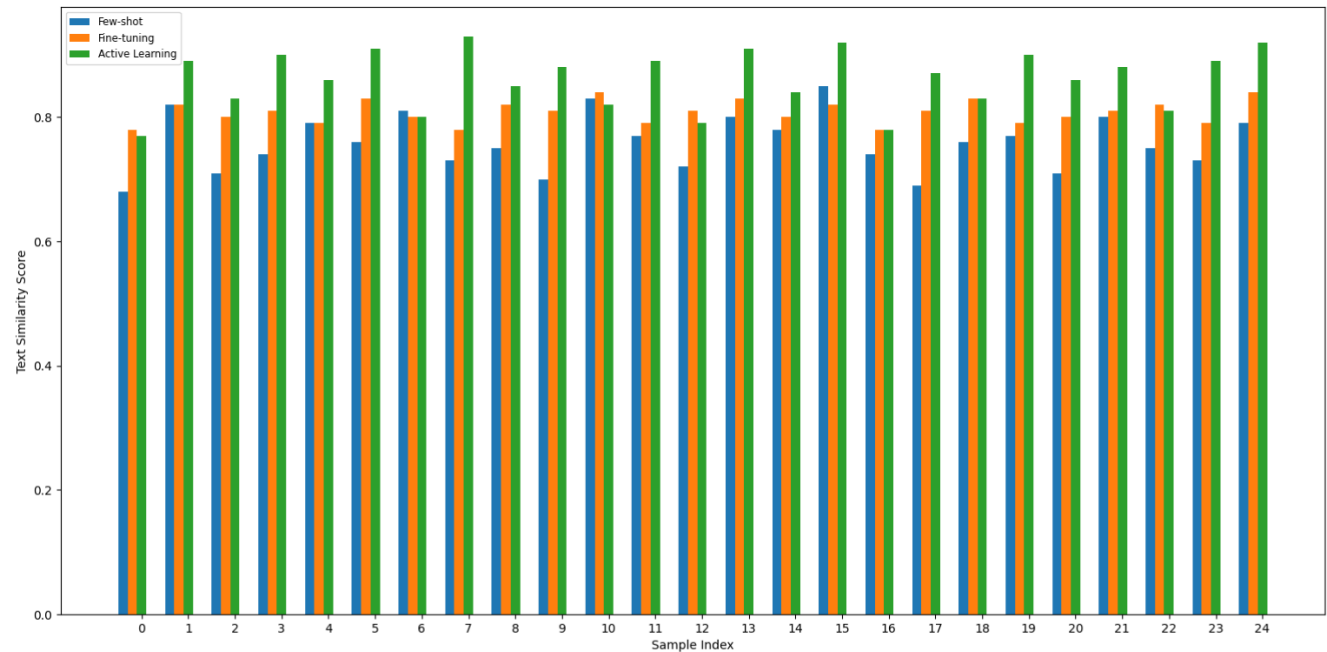


FIGURE 10. Comparing the text similarity accuracy scores for the different learning methods.

accuracies through iterative refinement and expert feedback integration.

1) COMPUTATIONAL INNOVATIONS AND PRACTICAL IMPLICATIONS FOR AUTOMATED COMPLIANCE CHECKING

Integration of fine-tuning with progressive active learning in our study represents a significant computational advancement

in the automated handling of complex regulatory texts. This approach effectively addresses key challenges in automating RASE annotation by combining domain-specific model adaptation with iterative expert feedback. This dual strategy enhances the model’s accuracy and efficiency, particularly in processing intricate, nested rules within regulatory documents. Additionally, the scalability and adaptability of our

method ensure its applicability across various regulatory contexts. From an engineering perspective, the ability to automate RASE annotation through this approach can significantly streamline regulatory compliance checking within the Architecture, Engineering, and Construction (AEC) industry. The method can be integrated into existing BIM systems, enabling real-time compliance checks that reduce manual effort and minimise the risk of costly errors during the design and construction phases. This scalability also suggests broader potential for adoption across the industry.

F. LIMITATIONS AND FUTURE WORK

The proposed system demonstrates promising capabilities for automating building regulatory compliance, yet it faces several limitations. Firstly, the model's performance is highly dependent on the quality and diversity of training data. Limited or imperfect datasets can lead to significant inaccuracies, particularly when handling complex or nuanced regulations. The need for human experts to provide regular oversight, especially in unusual or intricate regulatory scenarios, highlights the current reliance on manual interventions to ensure accuracy. Compatibility challenges with existing BIM platforms and the need for translation layers also present significant hurdles to seamless integration, alongside potential performance concerns when managing large datasets.

To address these limitations, future work should focus on enhancing the system's robustness through continuous fine-tuning with a growing, high-quality dataset. Integrating symbolic reasoning or rule-based AI elements could help the model better handle exceptions and nested regulations, thereby reducing the dependence on human feedback. Moreover, developing standardized data formats or custom APIs could improve interoperability with BIM systems, making integration more efficient. Expanding the model's adaptability to diverse regulatory environments through targeted regional fine-tuning and further refining human-in-the-loop strategies will also be crucial in reducing oversight while maintaining accuracy. These efforts are aimed at improving the system's scalability and reducing the human burden without compromising the quality of compliance checks.

V. CONCLUSION

This study presents a comprehensive approach to automating the Requirement, Applicability, Selection, and Exception (RASE) annotation process for building regulatory compliance using Large Language Models (LLMs) and active learning techniques. The key contributions include the demonstration of few-shot learning, fine-tuning, and progressive active learning methodologies, each enhancing the accuracy and efficiency of transforming complex regulatory texts into structured, machine-processable formats like YAML. These methods, especially the integration of human-in-the-loop learning, show promising results in mitigating the

challenges of sparse and noisy data, improving the semantic and structural quality of automated compliance outputs.

Despite these advancements, significant challenges remain. Issues such as dependency on high-quality datasets, the need for further human oversight, and compatibility with existing Building Information Modelling (BIM) systems indicate that there is more work to be done. Addressing these challenges will require not only technological improvements but also strategic integration of standardised data formats and broader applicability across different regulatory contexts.

Future research should focus on refining the model's adaptability to diverse regulatory environments, improving interoperability with BIM, and enhancing the automation of complex rule identification. Expanding the use of symbolic reasoning or rule-based AI elements could further bolster accuracy while reducing reliance on human oversight. Ultimately, these efforts could streamline automated compliance checking within the Architecture, Engineering, and Construction (AEC) industry, significantly reducing manual effort and associated risks.

The primary takeaways of this study demonstrate the potential of combining LLMs with active learning to automate compliance processes effectively. The findings indicate a future where automated compliance checking in the AEC industry becomes more robust, reducing errors and fostering efficiency in design and construction workflows.

REFERENCES

- [1] E. Hjelseth and N. N. Nisbet, "Capturing normative constraints by use of the semantic mark-up RASE methodology," in *Proc. CIB Conf.*, Jan. 2011, pp. 1–10.
- [2] N. Nisbet, L. Ma, and G. Aksenova, "Presentations of rase knowledge mark-up," in *Proc. Comput. Construct.*, Jul. 2022, pp. 184–190.
- [3] O. Ben-Kiki, C. Evans, and B. Ingerson. (2009). *YAML Ain't Markup Language (YAML) Version 1.1*. [Online]. Available: <https://yaml.org/spec/history/2004-12-28/2004-12-28.pdf>
- [4] A. Dridi, E. Vakaj, P. Patlakas, H. Hettiarachchi, T. Beach, J. Yeung, G. Costa, P. Hradil, and H. Tan, "D2.2 BCO ontology and rules format," IntechOpen Ltd., London, U.K., Tech. Rep. D2.2, 2023.
- [5] M. Aydin. (2022). *A Review of BIM-based Automated Code Compliance Checking: A Meta-Analysis Research*. [Online]. Available: <https://www.intechopen.com/chapters/12345>
- [6] F. Noardo, D. Guler, J. Fauth, G. Malacarne, S. M. Ventura, M. Azenha, P.-O. Olsson, and L. Senger, "Unveiling the actual progress of digital building permit: Getting awareness through a critical state of the art review," *Building Environ.*, vol. 213, Apr. 2022, Art. no. 108854.
- [7] C. Sydora and E. Stroulia. (2019). *Towards Rule-Based Model Checking of Building Information Models*. [Online]. Available: https://www.iaarc.org/publications/2019/Towards_Rule-Based_Model_Checking_of_Building_Information_Models
- [8] T. H. Beach, J.-L. Hippolyte, and Y. Rezgui, "Towards the adoption of automated regulatory compliance checking in the built environment," *Autom. Construct.*, vol. 118, Oct. 2020, Art. no. 103285.
- [9] H. Zhao, A. Kosmrlj, and S. S. Datta, "Chemotactic motility-induced phase separation," 2023, *arXiv:2301.12345*.
- [10] M. Jones, K. Kang, and S. Vandekar, "RESI: An R package for robust effect sizes," 2023, *arXiv:2302.12345*.
- [11] Q. Ding, D. Ding, Y. Wang, C. Guan, and B. Ding, "Unraveling the landscape of large language models: A systematic review and future perspectives," *J. Electron. Bus. Digit. Econ.*, vol. 3, no. 1, pp. 3–19, Feb. 2024.
- [12] X. V. Lin, "Few-shot learning with multilingual language models," *J. Artif. Intell. Res.*, vol. 1, pp. 1–14, Apr. 2023.

- [13] J.-Q. Jiang, G. Ye, and Y.-S. Piao, "Impact of the Hubble tension on the r - n_s contour," 2023, *arXiv:2303.12345*.
- [14] D. Al-Turki, M. Kyriakou, S. Basurra, M. M. Gaber, and M. M. Abdelsamea, "The power of progressive active learning in floorplan images for energy assessment," *Sci. Rep.*, vol. 13, no. 1, pp. 1–12, Sep. 2023.
- [15] M. Fu, T. Yuan, F. Wan, S. Xu, and Q. Ye, "Agreement-discrepancy-selection: Active learning with progressive distribution alignment," *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 8, pp. 7466–7473, May 2021.
- [16] Z. Cai, S. Zhang, P. Guo, J. Zhang, and L. Hu, "A progressive stacking pseudoinverse learning framework via active learning in random subspaces," *IEEE Trans. Syst. Man, Cybern. Syst.*, vol. 54, no. 5, pp. 2822–2832, May 2024.
- [17] Y. Bai, H. Ding, S. Bian, T. Chen, Y. Sun, and W. Wang, "SimGNN: A neural network approach to fast graph similarity computation," in *Proc. 12th ACM Int. Conf. Web Search Data Mining*, Jan. 2019, pp. 384–392.
- [18] S. Fuchs and R. Amor, "Natural language processing for building code interpretation: A systematic literature review," in *Proc. Conf. CIB W78*, 2021, pp. 11–15.
- [19] Z. Zhang, L. Ma, and N. Nisbet, "Unpacking ambiguity in building requirements to support automated compliance checking," *J. Manage. Eng.*, vol. 39, no. 5, pp. 1–10, Sep. 2023.
- [20] (2024). *OpenAI Fine-Tuning Guide*. [Online]. Available: <https://platform.openai.com/docs/guides/fine-tuning>
- [21] K. Poulinakis, D. Drikakis, I. W. Kokkinakis, and S. M. Spottswood, "Machine-learning methods on noisy and sparse data," *Mathematics*, vol. 11, no. 1, p. 236, Jan. 2023, doi: [10.3390/MATH11010236](https://doi.org/10.3390/MATH11010236).
- [22] E. Barbierato and A. Gatti, "Title not specified," *Electron*, vol. 13, no. 416, pp. 1–19, 2024.



ACCORD. His research interests include computer vision and AI, including deep learning, active learning, image processing, and large language model.

DHOYAZAN AL-TURKI is currently pursuing the Ph.D. degree with Birmingham City University. He was the official speaker of Microsoft, Jordan, from 2016 to 2018, and was invited as a speaker in Tedx, Jordan, in 2020 (AI and mysterious future). He is an IT professional with more than 16 years of experience in the field of web, mobile, and AI apps development, for many governmental and private sectors. He was engaged in several European and U.K. funded projects, such as Ecrofit and



Portsmouth, as a Senior Lecturer of computer science, both in U.K. He has published over 200 articles, co-authored three monograph-style books, and edited/co-edited six books on data mining and knowledge discovery. His work has attracted well over 8000 citations, with an H-index of 43. He served in the program committees of major conferences related to data mining, including ICDM, PAKDD, ECML/PKDD, and ICML. He is recognized as a fellow of British Higher Education Academy (HEA). He is also a member of the International Panel of Expert Advisers for the Australasian Data Mining Conferences. In 2007, he was awarded the CSIRO Teamwork Award. He also co-chaired numerous scientific events on various data mining topics.

MOHAMED MEDHAT GABER received the Ph.D. degree from Monash University, Australia. He then held appointments with The University of Sydney, CSIRO, and Monash University, all in Australia. He is currently a Professor of data analytics with the School of Computing and Digital Technology, Birmingham City University. Prior to joining Birmingham City University, he was with Robert Gordon University, as a Reader of computer science, and with the University of



Mechanochemical Cell Biology, Warwick University, Nottingham Molecular Pathology Node (NMPN), and Division of Cancer and Stem Cells both at Nottingham Medical School, as a Research Fellow. In 2016, he was a Marie Curie Research Fellow with the School of Computer Science, Nottingham University. He is also a fellow with British Higher Education Academy. His research interests include computer vision including image processing, deep learning, data mining and machine learning, pattern recognition, and image analysis.

MOHAMMED M. ABDELSAMEA received the Ph.D. degree (with Doctor Europaeus) in computer science and engineering from the IMT Institute for Advanced Studies, Lucca, Italy. He is currently a Senior Lecturer of computer science with the University of Exeter. Before joining the University of Exeter, he was an Assistant Professor of data and information science with the School of Computing and Digital Technology. He was with the School of Computer Science, Nottingham University,



natural language processing (NLP) tasks, particularly focusing on societal and human security and safety, low-resource languages, multilingual models, and explainable ML. She served as an organizer for CASE shared tasks and workshops, a Program Committee Member for several NLP conferences and workshops, including RANLP, NLP4ICS, and NLP4KGC, and a Reviewer for several prestigious journals and conferences, including the *Journal of Natural Language Engineering* and LREC-COLING.

HANSI HETTIARACHCHI received the Ph.D. degree in computer science from Birmingham City University, in 2023. She is currently a Lecturer of security and protection science, computing and communications with Lancaster University, U.K. Before joining Lancaster University, she was an Assistant Lecturer with the School of Computing and Digital Technology, Birmingham City University. Her research primarily aims to develop machine learning (ML) approaches for



Carbon Laboratory, Birmingham City University. He recently joined the Computer Science Centre, as a Senior Lecturer of software engineering with Birmingham City University. His research interests include simulation and emulation of networks (vehicular, mesh, and sensor ad hoc network), game theory, multiagent systems, multiobjective optimization, model calibration, and dynamic simulation of zero carbon design and retrofit of buildings.

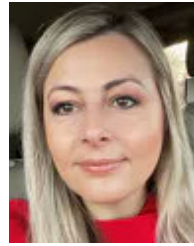
SHADI BASURRA received the B.Sc. degree (Hons.) in computer science from the University of Exeter, U.K., the M.Sc. degree in distributed systems and networks from the University of Kent, Canterbury, U.K., and the Ph.D. degree from the University of Bath in collaboration with the University of Bristol. After completing the Ph.D. degree, he was with Sony Corporation, developing goal decision systems, he then moved on to work as a Research Fellow with the Zero



SIMA IRANMANESH received the M.Sc. degree in software engineering from Yazd University, Iran, and the Ph.D. degree in computer science from the University of Warwick, U.K. She is currently a Lecturer of data science with the School of Computing and Digital Technology, Birmingham City University, U.K. Her research interests include search algorithms for large-scale graph data, graph-based similarity algorithms, and recommendation systems. She is also exploring applications of large language models (LLMs), particularly utilizing retrieval augmented generation (RAG) and knowledge graph technologies. Additionally, she is an active reviewer for various journals and conferences in the field of data science.



HADEEL SAADANY is currently a Lecturer of data science and natural language processing (NLP) with Birmingham City University. She conducts research in multidisciplinary fields focusing on the intersecting areas of NLP, deep learning, and artificial intelligence (AI) in different domains. Her research projects are primarily focused on utilizing AI to solve real-life problems through collaborative research with U.K. and USA industrial partners. She has previously worked at the University of Surrey, as a Postdoctoral Research Fellow, where she acted as a Lead Researcher in a research-industrial project between eBay and the Institute for People-Centered Artificial Intelligence (PAI). She employed recent Large Language Model AI technology to improve the efficacy of the infrastructure of the search engine for eBay cooperation. She has been the Lead Researcher of an Innovate U.K. Research-Industrial Project, where she applied NLP techniques for building a multi-modal user interface (UI) system that facilitates legal information acquisition for U.K. Supreme Courts. The UI system is currently in the application process to be published as an innovative patent by U.K. Intellectual Property Office.



EDLIRA VAKAJ is currently leading the Research and Innovation, College of Computing, and an Associate Professor of neuro-symbolic AI with Birmingham City University. She conducts research in multidisciplinary projects focusing on semantic web, knowledge graphs, AI, and semantic data spaces. She is the Principal Investigator of the ACCORD Horizon Project and engaged in several European and U.K. funded projects of various domains where semantic web technologies are applied, such as renewable energy (FP7 RENESENG), industrialized construction, and Industry 4.0 (Innovate U.K. DfMA and Knowledge Transfer Partnership), and higher education and youth (Erasmus + Capacity Building, Learning Mobility of Individuals, Cooperation for Innovation and the Exchange of Good Practices Action). She is acting as the Academic Lead of several knowledge transfer partnership (KTP) projects, such as the 4Net KTP and Hadley Group KTP. She is a Professional Member of British Computer Society; a fellow and AURORA of the Higher Education Academy, U.K.; and a Marie Curie Experienced Researcher Fellow of the University of Surrey. She is an Active Member of various communities, such as the Alan Turing Knowledge Graph Community, Linked Building Data, Knowledge, Graph Creation, and Common Action.

...